

1.11. Effacer un Mot fait de variables

Le même Mot, effacé par son seul nom.

1. Ce qu'il faut comprendre

`effacer(mot_xy)` relit `x`, `y` et `MOT` : il efface exactement ce que `texte(mot_xy)` a écrit.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Mot>` → [1.7. Mot — un texte et sa place, sous un nom](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction `effacer\(\)` — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **377 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage	– la mise en route de la console
main()	40 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code>	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> , <code>EcrireTexte</code>	– attend de pouvoir écrire une case
AdresseCase	10 o	← <code>texte()</code> , <code>effacer()</code>	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code>	– efface des cases
EcrireTexte	10 o	← <code>texte()</code>	– écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	96 o	← <code>CopierTuiles</code>	– 6 lettres sur 43
"BONJOUR"	7 o	← <code>texte()</code>	– un texte gravé

Lettres gravées : BJNORU

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>texte()</code>	× 1	18 o sur place, dans <code>main()</code>	+ <code>AdresseCase</code> (10 o), <code>EcrireTexte</code> (10 o)
<code>effacer()</code>	× 1	15 o sur place, dans <code>main()</code>	+ <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code>	+ <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Mot>          // un texte et sa place, sous un seul nom
#include <texte>         // écrit un texte à l'écran
#include <effacer>       // efface des cases, ou tout le fond
```

2. UNE VARIABLE — X

```
uint8_t x = 5;
```

3. UNE VARIABLE — Y

```
uint8_t y = 6;
```

4. UN TEXTE NOMMÉ — MOT

```
const char MOT[] = "BONJOUR";
```

5. UNE VARIABLE — MOT_XY

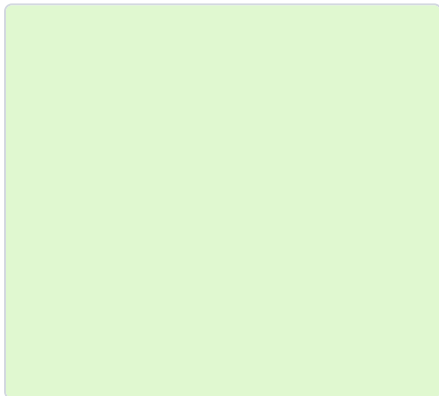
```
Mot mot_xy = { x, y, MOT };
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(mot_xy);  
    effacer(mot_xy);  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

« BONJOUR » est écrit puis effacé en (x, y) : l'écran reste vide.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ x vaut 5 et y vaut 6

✓ les sept cases de BONJOUR sont vides

✓ et l'écran est allumé

Ensuite : **1.12. Les variables dans leur propre fichier** — chapitre 1 : Les tout premiers pas. · gameboy3 — écrire une cartouche
Game Boy en C++